

1 Introduction:

Le but de ce jeu est de regarder une chaîne de caractère qui apparaît et la mémoriser le temps du niveau. Le jeu peut paraître simple aux premiers abords, mais la difficulté augmente avec les niveaux et les chapitres.

J'ai choisie de créer ce jeu car quand j'étais au lycée j'avais créé un jeu similaire sur ma calculatrice (une Ti-82) et malheureusement après avoir changée les piles, le jeu s'est effacé. Lorsqu'on m'a proposé de coder un jeu, j'ai tout de suite sauté sur l'occasion pour faire renaître ce jeu mais avec une meilleure version de celle-ci.

2 Installation et exécution du programme:

Pour lancer le jeu il suffit de compiler le jeu comme d'habitude et juste de rajouter à la toute fin "-lncurses" comme montré dans l'exemple ci-joint (car le jeu est codé en Incurses). Ces mots sont donc nécessaires pour le bon fonctionnement du jeu.

```
12205895@g212-5:~/prog1/projet_jeu$ gcc -Wall jeu.c -lncurses
12205895@g212-5:~/prog1/projet_jeu$ ./a.out
```

3 Présentation du jeu:

Les règles sont simples: un texte sera affiché sur votre écran, puis l'instant d'après plusieurs réponses s'afficheront à leur tour. Seule 1 réponse est correcte. Si vous parvenez à trouver la bonne réponse, vous passez au niveau suivant jusqu'au 15^{ème} niveaux (18 si vous comptez les niveaux bonus). En revanche, si malheureusement vous ne parvenez pas à trouver la bonne réponse, alors une fonction «vie» s'activera vous faisant perdre 1 vie ainsi qu'affichant un message spécifique à votre nombre de vie, et si vous n'avez plus de vie alors vous avez perdu.

4 Organisation du code:

Concernant les fonctions, vous pourrez en retrouver plusieurs:

-int afficher_entre(char phrase[]) => prend en entrée une chaîne de caractère phrase[], et qui renvoie alors les coordonnées du milieu de la colonne.

-void car() => pour compte à rebours, fait un décompte avant de lancer le niveau de sorte que le joueur se prépare mentalement pour le début du niveau.

-void vie(int *nb_vie) => qui prend en entrée le pointeur nb_vie. Cette fonction est appelée chaque fois que le joueur fait une erreur et elle fait 2 choses; d'abord elle enlève une vie, puis en fonction du nombre de vie elle affiche un message spécifique au nombre de vie du joueur. Si jamais le joueur n'a plus de vie lors de sa prochaine erreur, alors le joueur a perdu le jeu, et le programme s'arrête.

-void niv_suiv(int niveau) => prend en entrant un l'entier niveau pour afficher à quel niveau se situe le joueur, puis cette fonction attend que le joueur soit prêt avant de lancer le niveau qui va suivre.

-Chaque niveau prend les mêmes entrées c'est à dire *nb_vie ainsi que *niveau. Le pointeur nb_vie est pris ici afin que la fonction vie puisse prendre à son tour le nombre de vie du joueur et l'actualiser si il fait une erreur. Le pointeur niveau est pris ici afin que lorsque le joueur réussit le niveau, le niveau augmente de 1. Chaque niveau a une composition plus ou moins similaire, puis lors des derniers niveaux de chaque chapitre la difficulté devient alors beaucoup plus visible de part l'utilisation de différentes mécanique. Par exemple, texte à des positions différentes, l'utilisation de coloration, différente police utilisé ou encore affichage en clignotement manuel car certaines fois A_BLINK ne fonctionnait pas.

-void nb_aleas_tab(int nb[], int taille, int valeur) => Cette fonction n'est utilisé que dans le niveau bonus 2. Cette fonction prend en entrant le tableau nb[], l'entier taille et valeur. Le tableau est utilisé afin de stocker les différents nombres générés aléatoirement des 4 réponses. L'entier taille permet de savoir combien de valeur aléatoire vont être générés. L'entier valeur permet de savoir la valeur maximale que nous voulons avoir allant alors de 0 à valeur.

-int nb_aleas(int valeur) => Cette fonction n'est utilisé que dans le niveau bonus 2. Cette fonction prend en entrant un entier valeur, et retourne un nombre aléatoire de valeur maximale valeur. Cette fonction est utilisé afin de savoir quel sera le chiffre de la bonne réponse.

-void fin() => Cette fonction n'est utilisé qu'à la toute fin du jeu lorsque le joueur a réussi tous les niveaux, y compris les niveaux bonus. Elle affiche un message de victoire ainsi qu'une petite danse, puis ferme le programme.

5 Bug et TODO:

J'aurais bien apprécié pouvoir zoomer ou augmenter la taille des textes si possible pour que l'expérience de jeu soit la même pour tout le monde et pas qu'elle diffère en fonction de la taille de l'écran.

6 Remerciements et sources de documentation:

Je tiens à remercier plusieurs personnes, à commencer par Mr Pierre Rousselin pour ses cours ainsi que les réponses à mes différentes questions, puis Mickaël Zheng et Luila Lunzitis du groupe 4 en Pam 2 pour leur aide, ainsi que plusieurs de mes ami(e)s d'avoir testé le jeu et pour leur retours sur ce qu'il faudrait améliorer selon eux.

Concernant les sources qui m'ont permis de réussir ce jeu:

- https://arnaud-feltz.developpez.com/tutoriels/ncurses/?page=premier_pas
- https://www.youtube.com/watch?v=D-vx1_1Na3w
- https://www.mkssoftware.com/docs/man3/curs_attr.3.asp

- https://zestedesavoir.com/tutoriels/755/le-langage-c-1/1043_aggregats-memoire-et-fichiers/4277_les-pointeurs/
- <https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c/7672401-envoyez-des-pointeurs>
- http://www.linuxfocus.org/Francais//Archives/lf-2002_03-0233.pdf

7 Conclusion:

Personnellement je suis très fier de moi pour avoir réussi ce jeu au mieux. Bien que j'ai pas pu utilisé les structures car on a vu ce chapitre un peu tard dans l'année et mon jeu était terminé déjà, pour autant je trouve que mon jeu est assez complet comme ça.

8 Facultatif:

Ce jeu m'aura permit de voir l'étendu de mes capacités en informatique en trouvant réponses à mes problèmes, et bien que je ne suis pas très autonomes de nature, ce projet m'a beaucoup motivé à travailler régulièrement chez moi comme à l'université.